Intermediate (real branching) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper LangGraph, plus typing a routing function so a typo can't silently break it). Read Basic first even if you already know LangGraph — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

A plain edge (`add_edge`) always sends the graph to the same next node — there's no decision involved. A **conditional edge** adds a decision: after a node runs, a small function looks at the current state and picks which node runs next, out of a set of named options.

For this exercise: one node whose only job is deciding, a routing function that reads a `flag` field from state and returns a string naming a path, and two possible next nodes — one for each path.

Things to use:

- `graph.add_conditional_edges("from_node", routing_function, {"path_name": "target_node", ...})` — the mapping's keys are whatever strings your routing function can return; its values are the real node names to go to.
- Your routing function takes the whole state and returns one of the mapping's keys as a plain string.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

`add_conditional_edges` takes 3 arguments: the node the decision happens after, a function that reads state and returns a string, and a dict mapping each possible returned string to the real node it should lead to. The routing function itself is **not** a node — it doesn't get registered with `add_node`, and it doesn't return a state update. Its only job is picking a destination.

The exact pieces:

- `class GraphState(TypedDict): flag: bool; message: str` — a boolean field the routing function will branch on.
- `def route(state: GraphState) -> str:` with `if state["flag"]: return "path_a"`, then `return "path_b"` — reads state, returns a plain string naming the chosen path.
- `builder.add_conditional_edges("decide", route, {"path_a": "node_a", "path_b": "node_b"})` — wires it up: after node `"decide"` runs, call `route`, then go to whichever real node the returned string maps to.
- Both `node_a` and `node_b` still need their own `add_edge(..., END)` (or a further conditional edge) — a conditional edge only decides *where to go next*, it doesn't make either branch automatically finish the graph.
- Invoke twice, once with `{"flag": True, ...}` and once with `{"flag": False, ...}`, and confirm each one actually reaches the node you expect.

Think about what happens the moment `route` has a bug — say it returns `"path_c"` by mistake, a string that isn't a key in your mapping at all (this is exactly what the Edge cases exercise digs into). Plain Python gives you no protection here: `route`'s return type is just `str`, so nothing stops it from returning *any* string, including one your mapping was never built to handle.

The real design question isn't just "how do I wire a routing function" — it's "how do I make it hard for the routing function to return a value that isn't actually one of my real options?"

The extra piece that helps:

- `from typing import Literal` and `def route(state: GraphState) -> Literal["path_a", "path_b"]:` — this doesn't stop a bug at runtime by itself, but it does mean a type checker (or your editor) flags it immediately if you write `return "path_c"` or misspell `"path_a"` somewhere — catching the mistake while you're writing the code, not after you've run it and gotten a confusing failure.

This is worth doing any time a routing function's return value has to exactly match a fixed set of dict keys somewhere else in the file — the two are only connected by string-matching, and `Literal` is the cheapest way to make that connection visible to your tools instead of silent.

Difference between Basic and Intermediate: Basic writes `route` with a plain `-> str` return type, which is correct but gives no protection against a typo or an unhandled branch. Intermediate narrows that return type to `Literal["path_a", "path_b"]` — the exact set of keys the conditional edge's mapping actually understands — so a mismatch between what `route` can return and what the mapping can handle becomes a visible type error instead of a silent runtime surprise.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define a state shape: flag (true/false), message (text)

make node_decide: takes state, returns nothing new (just a pass-through)
make node_a: takes state, returns {"message": "went down path A"}
make node_b: takes state, returns {"message": "went down path B"}

make route(state): if state's flag is true, return "path_a";
                   otherwise return "path_b"

build the graph:
    add node_decide, node_a, node_b
    start -> node_decide
    from node_decide, use a conditional edge with route:
        "path_a" -> node_a
        "path_b" -> node_b
    node_a -> end
    node_b -> end

run it twice: once with flag=True, once with flag=False -- confirm
each one prints the right message
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# conditional_routing_practice.py — Intermediate section
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    flag: bool
    message: str

def node_decide(state):
    return {}

def node_a(state):
    return {"message": "went down path A"}
```

```
def node_b(state):
    return {"message": "went down path B"}

def route(state):
    if state["flag"]:
        return "path_a"
    return "path_b"

builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
```

Expected output if you run just this: nothing — add the edges (including the conditional one), `compile()`, and two `invoke(...)` calls with `print(...)` to see each path's message.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

define:
    class GraphState(TypedDict):
        flag: bool
        message: str

define:
    def node_decide(state: GraphState) -> dict:
        return {}

    def node_a(state: GraphState) -> dict:
        return {"message": "went down path A"}

    def node_b(state: GraphState) -> dict:
        return {"message": "went down path B"}

    def route(state: GraphState) -> str:
        if state["flag"]:
            return "path_a"
        return "path_b"

build:
    builder = StateGraph(GraphState)
    builder.add_node("node_decide", node_decide)
    builder.add_node("node_a", node_a)
    builder.add_node("node_b", node_b)
    builder.add_edge(START, "node_decide")
    builder.add_conditional_edges(
        "node_decide", route, {"path_a": "node_a", "path_b": "node_b"},
    )
    builder.add_edge("node_a", END)
    builder.add_edge("node_b", END)

run:
    graph = builder.compile()
    print(graph.invoke({"flag": True, "message": ""}))
    print(graph.invoke({"flag": False, "message": ""}))
```

Trace it by hand before running: with `flag=True`, which node should run after `node_decide`? Then run it and compare against the Solution.

Once that's working, narrow `route`'s return type so a typo can't silently break it:

```
same graph as Intermediate, but:

from typing import Literal

def route(state: GraphState) -> Literal["path_a", "path_b"]:
    if state["flag"]:
        return "path_a"
    return "path_b"
```

Run a type checker (or just read carefully) over a *deliberately broken* version where the mapping only has `"path_a"` and `route` can still return `"path_b"` — confirm your editor or `mypy` flags the mismatch before you ever run the code. Then fix the mapping and confirm both branches still work exactly as before.

Difference between Basic and Intermediate: Basic's `route` returns a plain `str` — correct, but nothing connects it to the specific keys `add_conditional_edges` actually understands except your own care. Intermediate narrows that to `Literal["path_a", "path_b"]`, turning a purely runtime connection (string-matching) into something your tools can check before the code ever runs.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution